

An Efficient Aerial Small Object Detection Pipeline Using YOLO26l and EigenCAM

Urooj Murad, Karthik M. Swamy and Chun Lo *Member,*

IEEE

Abstract Small object detection in aerial imagery remains a challenging problem in computer vision due to limited object resolution, background complexity, and feature loss during deep network processing. These challenges become more severe in drone-captured imagery, where objects such as pedestrians and vehicles often occupy only a small number of pixels within high-density scenes. This work proposes an efficient pipeline for small object detection in drone-captured aerial images using the YOLO26l object detection framework fine-tuned on the VisDrone dataset. The proposed approach combines high-resolution training, EigenCAM-based spatial attention analysis, threshold-based clustering, and TensorRT deployment optimization to improve both interpretability and real-time inference performance. EigenCAM heatmaps are utilized to identify dense object regions and analyze the attention behavior of the model in cluttered scenes. Furthermore, TensorRT optimization is applied to accelerate inference for practical deployment in real-time surveillance applications. Experimental results on the VisDrone validation dataset demonstrate that the fine-tuned YOLO26l model achieves a precision of 0.670, recall of 0.593, mAP50 of 0.604 and mAP50-95 of 0.384 while maintaining an inference latency of 4.3 ms per image on an NVIDIA RTX 4080 SUPER GPU. In addition to quantitative performance, the proposed framework provides qualitative insights into model attention behavior without requiring additional manual annotations. The results indicate that the proposed pipeline provides an effective balance between detection accuracy, computational efficiency, interpretability, and deployment feasibility for aerial small object detection tasks.

Keywords: Small object detection, aerial imagery, drone surveillance, YOLO26l, EigenCAM, TensorRT optimization, computer vision, deep learning, real-time object detection, VisDrone dataset.

I. INTRODUCTION

Object detection is one of the most important tasks in computer vision, focusing on identifying and localizing objects within images. There are two primary components of this process: classification, which determines the category of the object, and localization, which identifies the location of the

object using bounding boxes. In modern object detection, deep learning techniques like convolutional neural networks (CNNs) and region-based methods are commonly used. A high level of accuracy has been achieved in detecting vehicle, pedestrian, and animal objects. Object detection has been widely applied in real-world settings, such as autonomous driving, surveillance, and medical imaging [1] [2]. A rapid advancement in deep learning has made object detection models significantly more accurate and faster. CNNs have made it possible to build powerful detection frameworks among these. As a result of their strong performance in real-world scenarios, modern detectors such as YOLO, SSD, and Faster R-CNN have become widely adopted. Previous studies have shown that deep learning and large-scale datasets can significantly increase detection performance, as demonstrated by foundational works such as Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks and SSD: Single Shot MultiBox Detector [3] [4].

The simplicity of OLO (You Only Look Once) and its real-time performance make it widely used. In contrast to traditional two-stage methods, YOLO formulates object detection as a single regression problem, making it more efficient. Despite these advantages, YOLO and similar detectors have trouble detecting tiny objects [5] [6]. Despite these advances, computer vision faces significant challenges when it comes to detecting tiny objects. Computer vision still faces significant challenges when it comes to detecting tiny objects despite these advances. A tiny object, usually smaller than 32 pixels, lacks much visual information and is highly susceptible to noise, occlusion, and complex backgrounds. A previous study of Feature Pyramid Networks highlighted the difficulty of representing nano-scale features in deep neural networks. [7] [8].

Another major challenge in small object detection is the imbalance between foreground objects and background regions within aerial images [9]. Due to the limited size of tiny objects, important semantic information can easily be obscured by surrounding background features when determining feature extraction [7]. This issue often causes a problem where multiple small objects appear close to each other, which makes it harder for the localization to be accurate, particularly in crowded urban areas. Consequently, modern object detection systems must preserve discriminative spatial information at

different feature scales [10].

With high-altitude views, pedestrians and vehicles appear extremely small, making aerial imagery even more challenging. When deep neural networks are downsampled, they may lose important features that may lead to reduce detection accuracy [8] [11]. In order to overcome these challenges, recent research has focused on techniques such as multi-scale feature learning, attention mechanisms, and data augmentation strategies. However, many of these approaches require architectural changes or increase the complexity of models [12] [13].

The increasing use of unmanned aerial vehicles (UAVs) and drone-based imaging systems has also increased the need for intelligent and automated visual analysis solutions [14]. Many applications, such as traffic monitoring, disaster response, border surveillance, and urban management, depend heavily on the accurate detection of small targets from aerial viewpoints. Real-world applications, on the other hand, often operate with strict computational and latency constraints, which makes efficient deployment just as important as accurate detection [15].

Furthermore, in real-world deployment scenarios such as drone-based surveillance and traffic monitoring, both high detection accuracy and low-latency inference are required along with resource-constrained hardware. It introduces a critical trade-off between modeling complexity and computation efficiency. As a result, developing detection pipelines that are lightweight and deployable while maintaining competitive accuracy has become an important research focus. To continue, explainability of deep object detectors is gaining attention, as understanding their behavior in dense or cluttered scenes can help to identify failures and improve robustness. In complex environments, activation-based visualization methods provide insights into how models attend to small and overlapping objects [16] [17].

To improve model reliability and transparency, it has become increasingly important to understand the internal behavior of deep neural networks. The use of visualization and attention-analysis can help interpret which image regions make the most significant contributions to the detector's predictions. These interpretability techniques are particularly useful in aerial imagery, since dense object distributions and complex backgrounds complicate quantitative analysis [16] [17]. Therefore, there is a need for lightweight and efficient detection methods.

II. RELATED WORK

A. Object Detection Architectures and YOLO-based Models

In computer vision, object detection involves classifying and localizing objects within an image [18]. In early approaches, such as Faster R-CNN, region proposals are generated first, then classified, which achieves high detection accuracy, but is computationally expensive for real-time applications. As an alternative to this, one-stage detectors such as YOLO and SSD provide real-time performance, making them a popular choice for practical applications. [19]. In recent versions of

YOLO, accuracy and inference efficiency across different hardware platforms have been improved significantly [20] [21]. A number of recent studies on YOLO-based models demonstrate continuous improvements due to architectural refinements and optimization strategies, particularly when it comes to balancing speed and accuracy for scenarios like autonomous systems and surveillance. [4] [22] [23]. In recent research, lighter and more efficient YOLO variants have also been explored for embedded and edge devices [24]. In these studies, computational complexity is reduced while competitive detection performance is maintained, usually through architectural simplification and optimized feature extraction techniques. Despite these accomplishments, it remains a challenge to achieve high accuracy, especially for small objects. [25] [24] [23]. In more recent developments, YOLO-based frameworks are being integrated with hardware-aware optimization techniques, enabling GPUs and edge accelerators to be deployed with significantly reduced latency [15]. In real-time systems, inference speed is as critical as detection accuracy for inference. Additionally, YOLO research has increasingly focused on visualizing features and improving training stability through advanced optimization strategies. By utilizing improved label assignments, adaptive loss functions, and efficient feature aggregation modules, localization accuracy and generalization have been improved across diverse datasets [26]. As a result of these developments, YOLO-based detectors remain competitive in both industrial and research applications while preserving their real-time inference capabilities.

B. Small and Tiny Object Detection in Complex Scenes

Small object detection remains one of the most challenging problems in computer vision due to the limited number of pixels representing the object and the loss of discriminative features during deep network processing [27]. These objects often occupy a very small portion of the image, making them highly sensitive to noise, occlusion, and background clutter. As a result, they are highly sensitive to noise, occlusion, and background clutter. Consequently, standard detection models fail to accurately classify or locate them, particularly in complex environments such as remote sensing or aerial imagery [8] [9].

In order to overcome these challenges, various approaches have been proposed in the literature, such as multi-scale feature learning, feature pyramid networks, context-aware modeling, and super-resolution algorithms [7] [28]. Recent studies have examined attention-based mechanisms and transformer-based architectures for enhancing small object feature representation [12] [29]. Methods like these aim to improve the ability of detection networks to preserve fine-grained spatial information and to enhance multi-scale feature fusion by preserving spatial information at multiple scales [30] [31]. Because of scale variation and environmental complexity, detecting tiny objects in real-world scenarios remains challenging [8].

Additionally, many studies point out that aerial imagery presents unique challenges as compared with natural images,

including extreme object sparsity, high local object density, and strong background similarity. As a consequence, standard detectors trained on COCO datasets degrade significantly.

In addition to improving model interpretability for small object detection, spatio-attention analysis has also been investigated. The use of visualization approaches like Grad-CAM and EigenCAM has been investigated for better understanding how deep neural networks react to dense objects and cluttered backgrounds. Using these methods can provide qualitative insight into feature activation behavior and identify detection limitations that may not be adequately captured by quantitative evaluation metrics. To make safety-critical applications more reliable, such interpretability techniques are increasingly being incorporated into detection pipelines.

C. Model Optimization and Real-Time Deployment Acceleration

In addition to improving detection accuracy, recent research has increasingly focused on optimizing object detection models for real-time deployment. Techniques such as model compression, quantization, and hardware-specific optimization have been widely explored to reduce inference latency while maintaining accuracy. In particular, TensorRT-based optimization has become a common approach for accelerating deep learning inference on NVIDIA GPUs by converting models into highly optimized runtime engines [32].

Recent studies show that combining efficient model architectures with inference-level optimization techniques can significantly improve deployment performance in real-time systems. This is especially important for applications requiring fast processing speeds, such as autonomous navigation and surveillance systems, where maintaining a balance between accuracy and latency is critical. However, integrating such optimization techniques with detection improvements remains an active research area [33] [15].

Furthermore, recent works emphasize that end-to-end optimization pipelines-combining model architecture design, training strategies, and inference acceleration provide better real-world performance than optimizing each component independently. This motivates integrated frameworks that jointly consider accuracy, efficiency, and deployability.

Although substantial progress has been achieved in object detection, efficiently detecting small objects in aerial imagery while maintaining real-time deployment performance remains a challenging problem. Motivated by these limitations, this work proposes an integrated framework combining YOLO26l fine-tuning, attention-based analysis, and deployment optimization for efficient aerial small object detection.

III. METHODOLOGY

A. Overview

This work proposes a pipeline for small object detection in drone-captured aerial imagery consisting of three stages: (1) domain-specific fine-tuning of YOLO26l on the VisDrone dataset, (2) EigenCAM-based heatmap analysis combined with threshold-based clustering to identify and refine detection in

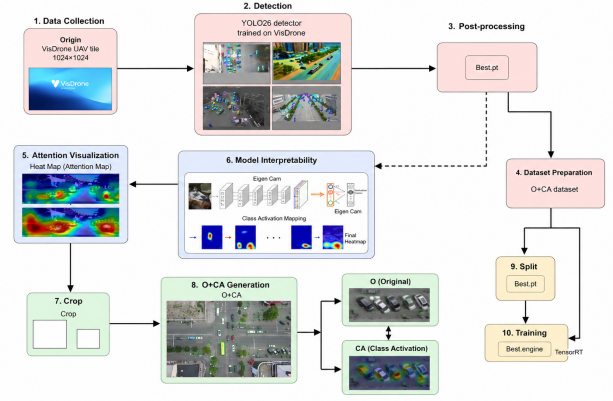


Fig. 1. Proposed framework for aerial small object detection and cross-dataset generalization

high-density regions, and (3) TensorRT-accelerated deployment for real-time inference evaluation. Formally, let an input image be denoted as $x \in \mathbb{R}^{H \times W \times 3}$. The pipeline learns a mapping $f_\theta : x \mapsto \hat{\mathcal{Y}}$, where $\hat{\mathcal{Y}} = \{(\hat{b}_i, \hat{c}_i, \hat{s}_i)\}_{i=1}^N$ represents the set of predicted bounding boxes $\hat{b}_i$, class labels $\hat{c}_i$, and confidence scores $\hat{s}_i$. Fig. ?? illustrates the overall pipeline.

B. Baseline Model and Fine-Tuning

YOLO26l, the large variant of the YOLO26 model family [1], was selected as the base model for this work. YOLO26l contains approximately 24.8 million parameters and was pre-trained on the COCO dataset. Formally, the model can be decomposed into a backbone feature extractor $\mathcal{B}_\theta$ and a detection head $\mathcal{H}_\theta$. Given an input image $x \in \mathbb{R}^{1024 \times 1024 \times 3}$, the backbone produces hierarchical feature maps:

$$F = \mathcal{B}_\theta(x), \quad F = \{F_3, F_4, F_5\}, \quad (1)$$

where $F_l \in \mathbb{R}^{H_l \times W_l \times C_l}$ denotes the feature representation at scale l . The detection head then produces dense predictions:

$$P = \mathcal{H}_\theta(F) \in \mathbb{R}^{S \times S \times (C+4+1)}, \quad (2)$$

where $S \times S$ is the spatial grid resolution, $C = 10$ is the number of VisDrone categories, the four regression terms encode bounding-box center offsets and dimensions, and the final scalar corresponds to objectness confidence.

Fine-tuning was performed on the VisDrone training split using an input resolution of 1024×1024 pixels and a batch size of 4 on an NVIDIA RTX 4080 GPU with 16GB VRAM. Training was conducted for 50 epochs using the default Ultralytics training configuration, with the CUDA memory allocator set to expandable segments to handle the high-resolution inputs efficiently.

Standard data augmentation was applied during training, including horizontal flipping, HSV color jitter, mosaic augmentation, and random scaling. No rotation or vertical flipping was applied, as aerial imagery exhibits a consistent top-down perspective with no vertical symmetry. The model is optimized using a composite loss function:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{bbox}} \mathcal{L}_{\text{CIoU}} + \lambda_{\text{obj}} \mathcal{L}_{\text{obj}} + \lambda_{\text{cls}} \mathcal{L}_{\text{cls}}, \quad (3)$$

where λ_{bbox} , λ_{obj} , and λ_{cls} are weighting coefficients. The bounding-box regression loss is defined as:

$$\mathcal{L}_{\text{CIoU}} = 1 - \text{IoU} + \frac{\rho^2(b, b_{\text{gt}})}{c^2} + \alpha v, \quad (4)$$

where $\rho(\cdot)$ denotes the Euclidean distance between the center points of the predicted box b and the ground-truth box b_{gt} , c is the diagonal length of the smallest enclosing box, and v measures aspect-ratio consistency with weight α . The classification and objectness losses are computed via binary cross-entropy:

$$\mathcal{L}_{\text{cls}} = \frac{1}{C} \sum_{c=1}^C \text{BCE}(\sigma(t_c), y_c), \quad (5)$$

$$\mathcal{L}_{\text{obj}} = \text{BCE}(\sigma(t_{\text{obj}}), y_{\text{obj}}), \quad (6)$$

where $\sigma(\cdot)$ is the sigmoid function, t_c and t_{obj} are the predicted logits, and $y_c, y_{\text{obj}} \in \{0, 1\}$ are the corresponding ground-truth labels.

The best model checkpoint, selected based on validation mAP50-95 achieved a precision of 0.670, recall of 0.593, mAP50 of 0.604 and mAP50-95 of 0.384 on the VisDrone validation set.

C. EigenCAM Heatmap Analysis

To interpret the spatial attention of the fine-tuned model and identify regions of high detection confidence, EigenCAM [34] is applied to the model's backbone feature maps. EigenCAM was selected over GradCAM++ due to its compatibility with the YOLO26 architecture, which does not expose intermediate gradients in a form suitable for gradient-based attribution methods.

Let $A \in \mathbb{R}^{K \times H' \times W'}$ denote the activation tensor extracted from the target convolutional layer of the backbone, where K is the number of channels and H', W' are the spatial dimensions. The tensor is reshaped into a two-dimensional matrix $\tilde{A} \in \mathbb{R}^{K \times (H'W')}$, where each column corresponds to a spatial activation vector. The covariance matrix is computed as:

$$\Sigma = \frac{1}{H'W'} \tilde{A} \tilde{A}^\top. \quad (7)$$

EigenCAM computes the first principal component of the feature-map activations by solving the eigenvalue problem $\Sigma v = \lambda v$. Let $v_1 \in \mathbb{R}^K$ denote the leading eigenvector corresponding to the largest eigenvalue λ_1 . The spatial heatmap $H \in \mathbb{R}^{H' \times W'}$ that highlights regions the model attends to most strongly during inference is obtained by projecting the reshaped features onto v_1 :

$$\tilde{H} = v_1^\top \tilde{A}, \quad H = \text{Reshape}(\tilde{H}, H', W'). \quad (8)$$

The resulting heatmap is upsampled to the input resolution via bilinear interpolation and normalized to the range $[0, 1]$ using min-max scaling:

$$H_{\text{norm}}(i, j) = \frac{H(i, j) - \min H}{\max H - \min H}. \quad (9)$$

These heatmaps are generated over the VisDrone validation set to provide qualitative insight into model behavior and to identify failure modes in cluttered or densely packed scenes.

D. Threshold-Based Clustering

The EigenCAM heatmaps are subsequently used to drive a threshold-based clustering stage. For each image, the normalized heatmap H_{norm} is binarized using a logically determined activation threshold τ , isolating high-response regions that correspond to areas of dense object presence:

$$B(i, j) = \begin{cases} 1, & \text{if } H_{\text{norm}}(i, j) > \tau, \\ 0, & \text{otherwise.} \end{cases} \quad (10)$$

Connected components within the binarized map are extracted and treated as candidate dense regions:

$$\mathcal{R} = \{r_1, r_2, \dots, r_m\} = \text{ConnectedComponents}(B), \quad (11)$$

where each region r_k is represented by its bounding coordinates. These regions are then used to guide focused re-examination of detection outputs within their boundaries, allowing the pipeline to specifically target areas where small objects are likely to cluster. This approach is motivated by the observation that in aerial imagery, objects such as vehicles and pedestrians tend to appear in spatially coherent groups, and leveraging the model's own attention signal provides a principled way to identify these regions without requiring additional annotations.

E. TensorRT Deployment

Following fine-tuning and analysis, the best model weights are exported to a TensorRT engine using the Ultralytics export pipeline. Let f_θ denote the trained detection network. TensorRT optimization constructs an optimized runtime graph f_{TRT} by fusing convolutional and batch normalization layers, eliminating redundant operations, and quantizing weights for the target hardware. The optimized engine satisfies:

$$\hat{Y} = f_{\text{TRT}}(x) \approx f_\theta(x), \quad (12)$$

with reduced inference latency $t_{\text{TRT}} \ll t_{\text{base}}$. YOLO26l's native end-to-end, NMS-free architecture [1] is particularly well-suited for TensorRT export, as the absence of a separate post-processing stage simplifies the computation graph and reduces export complexity. Inference latency and throughput are evaluated on the VisDrone validation set to assess the practical deployment performance of the fine-tuned model under real-time constraints.

IV. RESULTS

A. Quantitative Evaluation on VisDrone Dataset

The YOLO26l model was evaluated on the VisDrone validation set after fine-tuned, which is characterized by high-density scenes and extremely small object instances. As shown in Table I, the model achieved an overall precision of 0.670, recall of 0.593, mAP50 of 0.604 and mAP50-95 of 0.384. These results align with the performance benchmarks expected from large-scale variants of the YOLO26 architecture when applied to drone-captured imagery.

TABLE I
YOLO26L MODEL VALIDATION RESULTS ON VisDRONE DATASET

Model Info: 24.75M Parameters, 86.1 GFLOPs						
Speed: 4.3 ms inference per image (NVIDIA RTX 4080 SUPER)						
Class	Images	Instances	Box(P)	R	mAP50	mAP50-95
all	548	38759	0.670	0.593	0.604	0.384
pedestrian	520	8844	0.742	0.680	0.721	0.384
people	482	5125	0.728	0.541	0.591	0.266
bicycle	364	1287	0.583	0.402	0.431	0.225
car	515	14064	0.864	0.862	0.894	0.665
van	421	1975	0.620	0.625	0.598	0.444
truck	266	750	0.654	0.560	0.568	0.406
tricycle	337	1045	0.584	0.570	0.519	0.318
awning-tricycle	220	532	0.446	0.312	0.279	0.191
bus	131	251	0.773	0.706	0.737	0.574
motor	485	4886	0.708	0.671	0.696	0.363

B. Analysis of Category-Specific Performance

Analysis in detail of individual categories reveals that the model has a well performance on structured, especially in much larger objects. mAP50 of Cars is equal to 0.894 and buses is equal to 0.737. By contrast, there still have a large challenges in detecting smaller or more visually complex categories. Awning-tricycles mAP50 only get 0.279 and mAP50 of bicycles is 0.431. This performance gap is consistent with previous research indicating that objects with fewer pixels and limited discriminative features are highly susceptible to noise and background clutter in aerial views.

C. Inference Efficiency and Deployment

To meet the requirements for an real-time deployment in automated surveillance, the model was optimized by using the TensorRT runtime engine. Through NVIDIA RTX 4080 SUPER GPU, we obtain the average inference latency of 4.3ms per image after optimization of YOLO26l. The integration of YOLO26l's native NMS-free architecture with TensorRT's operator fusion reduced computation significantly, implementing an high-throughput processing suitable for edge arrangement.

D. Qualitative Insights via EigenCAM

Besides quantitative metrics, EigenCAM-based heatmap analysis was also implemented to interpret the model's spatial attention. The results indicate that the model is successfully focuses on the density of clusters of pedestrians and vehicles, validating the effectiveness of the fine-tuning process on high-resolution (1024×1024) inputs. These attention signals provide a principled basis for the subsequent threshold clustering stage, which specifically targets high-density regions without the need for additional manual annotations.

V. CONCLUSION

This work presented an efficient pipeline for small object detection in drone-based aerial imagery using a fine-tuned YOLO26l model on the VisDrone dataset. The study addressed the challenges of detecting small and densely distributed objects by combining high-resolution training, EigenCAM-based

attention analysis, threshold-based clustering, and TensorRT-based deployment optimization. Experimental results demonstrated that the proposed approach achieved competitive detection performance while maintaining real-time inference capability on GPU hardware.

The analysis further showed that the model performs effectively on larger and structured object categories such as cars and buses, while smaller categories remain more challenging due to limited visual features and complex backgrounds. EigenCAM visualization provided qualitative insights into the spatial attention behavior of the model, highlighting its ability to focus on dense object regions in aerial scenes. In addition, TensorRT optimization significantly reduced inference latency, making the system suitable for practical surveillance and edge deployment scenarios.

Overall, the proposed pipeline demonstrates a balanced approach between detection accuracy, interpretability, and computational efficiency for aerial small object detection. Future work may focus on improving tiny object representation through advanced feature fusion methods, transformer-based architectures, and adaptive clustering strategies to further enhance detection performance in highly complex environments.

The findings of this study demonstrate the potential of combining efficient object detection architectures with attention-based analysis and deployment optimization to support practical real-world aerial surveillance systems.

PROJECT REPOSITORY

The complete implementation, training scripts, experimental results, and supplementary materials are publicly available at: <https://github.com/KarthikSwamy129/Yolo26VisDroneAssignment>

VI. REFERENCES

- [1] G. Jocher and J. Qiu, "Ultralytics YOLO26," 2026. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [2] R. Nautiyal and M. Deshmukh, "Tiny object detection: An in-depth survey of techniques, challenges, and future directions," *Digital Signal Processing*, p. 105995, 2026.
- [3] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," *Advances in neural information processing systems*, vol. 28, 2015.
- [4] Z. Li, Y. Dong, L. Shen, Y. Liu, Y. Pei, H. Yang, L. Zheng, and J. Ma, "Development and challenges of object detection: A survey," *Neurocomputing*, vol. 598, p. 128102, 2024.
- [5] U. Nisa, M. S. M. Pozi, and M. A. Saip, "A decade of research in small object detection: a comprehensive bibliometric analysis," *International Journal of Data Science and Analytics*, vol. 20, no. 8, pp. 7331–7355, 2025.
- [6] H. Yu, J. Liu, and M. Lin, "A comprehensive literature review on yolo-based small object detection: Methods, challenges, and future trends," *Computers, Materials, & Continua*, vol. 87, no. 1, 2026.
- [7] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 2117–2125.
- [8] A. Aldubaikhi and S. Patel, "Advancements in small-object detection (2023–2025): Approaches, datasets, benchmarks, applications, and practical guidance," *Applied Sciences*, vol. 15, no. 22, p. 11882, 2025.
- [9] G.-S. Xia, X. Bai, J. Ding, Z. Zhu, S. Belongie, J. Luo, M. Datcu, M. Pelillo, and L. Zhang, "Dota: A large-scale dataset for object detection in aerial images," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 3974–3983.

TABLE II
MODEL EVALUATION COMPARISON TABLE WITH AUTOMATED BIBTEX CITATIONS

Method	Year&Venue	Baseline	Backbone	test data	#img	AP	AP ₅₀	AP ₇₅	AP _{small}	AP _{medium}	AP _{large}	s/img (GPU)
FRCNN [3]+FPN [7]	2015 NIPS	–	ResNet50	o	548	21.4	40.7	19.9	11.7	33.9	54.7	0.055
FRCNN [3]+FPN [7]	2015 NIPS	–	ResNet101	o	548	21.4	40.7	20.3	11.6	33.9	54.9	0.074
FRCNN [3]+FPN [7]	2015 NIPS	–	ResNeXt101	o	548	21.8	41.8	20.1	11.9	34.8	55.5	0.156
FRCNN [3]+FPN [7]+EIP	2015 NIPS	–	ResNet50	c	3,288	21.1	44.0	18.1	14.4	30.9	30.0	0.330
FRCNN [3]+FPN [7]+EIP	2015 NIPS	–	ResNet101	c	3,288	23.5	46.1	21.1	17.1	33.9	29.1	0.444
FRCNN [3]+FPN [7]+EIP	2015 NIPS	–	ResNeXt101	c	3,288	24.4	47.8	21.8	17.8	34.8	34.3	0.936
ClusDet [35]	2019 ICCV	FRCNN+FPN	ResNet50	o+ca	2,716	26.7	50.6	24.7	17.6	38.9	51.4	0.273
ClusDet [35]	2019 ICCV	FRCNN+FPN	ResNet101	o+ca	2,716	26.7	50.4	25.2	17.2	39.3	54.9	0.366
ClusDet [35]	2019 ICCV	FRCNN+FPN	ResNeXt101	o+ca	2,716	28.4	53.2	26.4	19.1	40.8	54.4	0.773
ClusDet* [35]	2019 ICCV	FRCNN+FPN	ResNeXt101	o+ca	2,716	32.4	56.2	31.6	–	–	–	0.773
DREN [36]	2019 ICCVW	MRCNN+FPN	ResNet101	–	–	27.1	–	–	–	–	–	–
DREN [36]	2019 ICCVW	MRCNN+FPN	ResNet152	–	–	30.3	–	–	–	–	–	–
DMNet [37]	2020 CVPRW	FRCNN+FPN	ResNet50	o+ca	2,736	28.2	47.6	28.9	19.9	39.6	55.8	0.29
DMNet [37]	2020 CVPRW	FRCNN+FPN	ResNet101	o+ca	2,736	28.5	48.1	29.4	20.0	39.7	57.1	0.36
DMNet [37]	2020 CVPRW	FRCNN+FPN	ResNeXt101	o+ca	2,736	29.4	49.3	30.6	21.6	41.0	56.7	0.61
CDMNet [38]	2021 ICCVW	FRCNN	ResNet50	ca	2,170	29.2	49.5	29.8	20.8	40.7	41.6	–
CDMNet [38]	2021 ICCVW	FRCNN	ResNet101	ca	2,170	29.7	50.0	30.9	21.2	41.8	42.9	–
CDMNet [38]	2021 ICCVW	FRCNN	ResNeXt101	ca	2,170	30.7	51.3	32.0	22.2	42.4	44.7	–
AMRNet [39]	2020 Arxiv	FRCNN+FPN	ResNet50	o+aug	–	31.7	–	–	23.0	43.4	58.1	–
AMRNet [39]	2020 Arxiv	FRCNN+FPN	ResNet101	o+aug	–	31.7	–	–	29.0	45.5	60.9	–
AMRNet [39]	2020 Arxiv	FRCNN+FPN	ResNeXt101	o+aug	–	32.1	–	–	23.2	43.9	60.5	–
GLSAN [40]	2021 TIP	FRCNN+FPN	ResNet50	o+ca	–	30.7	55.4	30.0	–	–	–	–
GLSAN [40]	2021 TIP	FRCNN+FPN	ResNet101	o+ca	–	30.7	55.4	30.0	–	–	–	–
GLSAN [40]	2021 TIP	Cascade R-CNN	ResNet50	o+ca	–	32.5	55.8	30.0	–	–	–	–
AFSM [26]	2020 Arxiv	CenterNet	CBResNet50	–	–	33.95	60.46	32.69	–	–	–	–
AFSM* [26]	2020 Arxiv	CenterNet	CBResNet50	–	–	37.62	65.41	37.06	–	–	–	–
AFSM [26]	2020 Arxiv	CenterNet	CBResNet50+DCN	–	–	35.43	61.88	34.60	–	–	–	–
AFSM* [26]	2020 Arxiv	CenterNet	CBResNet50+DCN	–	–	39.48	66.98	39.45	–	–	–	–
HRDNet [41]	2021 ICME	Cascade R-CNN	ResNet18+101	–	–	28.33	49.25	28.16	–	–	–	–
HRDNet* [41]	2021 ICME	Cascade R-CNN	ResNeXt50+101	–	–	35.51	62.00	35.13	–	–	–	–
FCOS+SAHI [42]	2022 ICIP	FCOS	–	ca	–	–	38.5	–	–	–	–	–
VFNet+SAHI [42]	2022 ICIP	VFNet	–	ca	–	–	42.2	–	–	–	–	–
TOOD+SAHI [42]	2022 ICIP	TOOD	–	ca	–	–	43.5	–	–	–	–	–
QueryDet [43]	2022 CVPR	RetinaNet	ResNet50	o	584	28.32	48.14	28.75	–	–	–	0.364
CEASC [44]	2023 CVPR	RetinaNet	ResNet50	o	584	20.8	35.0	27.7	–	–	–	0.070
CEASC [44]	2023 CVPR	GFL	ResNet18	o	584	28.7	50.7	28.4	–	–	–	0.046
CenterNet (Baseline) [45]	2019 Arxiv	–	Hourglass104	o	548	27.8	47.9	27.6	21.3	42.1	49.8	0.215
CenterNet [45]	–	CenterNet	HRNet	o	548	29.1	49.2	30.1	22.3	43.9	52.4	0.179
YOLC (k=0) [46]	–	CenterNet	HRNet	o	584	35.5	59.3	35.9	26.9	47.8	59.4	0.179
YOLC (k=1) [46]	–	CenterNet	HRNet	o+ca	1168	37.7	61.7	39.0	30.2	48.4	56.4	0.358
YOLC (k=2) [46]	–	CenterNet	HRNet	o+ca	1644	37.8	61.7	39.4	30.5	48.3	55.4	0.537
YOLC (k=3) [46]	–	CenterNet	HRNet	o+ca	2192	37.8	61.6	39.4	30.6	48.1	54.1	0.716
YOLC (k=3)* [46]	–	CenterNet	HRNet	o+ca	2192	38.3	62.3	40.1	31.7	48.0	46.5	0.956
CenterNet [46]	–	CenterNet	ResNet50	o	584	23.9	45.5	22.1	14.4	36.3	44.9	0.073
YOLC (k=0) [46]	–	CenterNet	ResNet50	o	584	28.9	51.4	28.3	20.1	41.6	47.3	0.136
YOLC (k=0) [46]	–	CenterNet	ResNet101	o	584	29.7	52.4	29.2	20.1	43.1	52.4	0.158
YOLC (k=2) [46]	–	CenterNet	ResNet50	o+ca	1168	31.8	55.0	31.7	24.7	42.3	45	0.441
YOLC (k=2) [46]	–	CenterNet	ResNet101	o+ca	1644	33.7	57.4	33.8	26.1	44.9	52.8	0.543
YOLC (k=2)* [46]	–	CenterNet	ResNet101	o+ca	1644	36.3	60.1	37.4	28.9	47.5	51.8	0.543
YOLC (k=2)* [46]	–	CenterNet	HRNet	o+ca	1644	39.6	63.7	41.6	32.8	49.4	56.7	0.537
YOLC (k=3)* [46]	–	CenterNet	HRNet	o+ca	2192	39.6	63.7	41.6	32.9	49.3	56.2	0.716
YOLO26L (Ours)	2026	YOLO26	YOLO26L	o	548	38.4	60.4	–	–	–	–	0.0043

- [10] J. Pang, C. Li, J. Shi, Z. Xu, and H. Feng, “ $\mathcal{R}^2$ -CNN: Fast tiny object detection in large-scale remote sensing images,” *IEEE Transactions on Geoscience and Remote Sensing*, vol. 57, no. 8, pp. 5512–5524, 2019.
- [11] H. Qiang, W. Hao, M. Xie, Q. Tang, H. Shi, Y. Zhao, and X. Han, “Scn-yolo for lightweight small object detection in remote sensing images,” *Remote Sensing*, vol. 17, no. 2, p. 249, 2025.
- [12] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, “Cbam: Convolutional block attention module,” in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 3–19.
- [13] M. Kisanal, Z. Wojna, J. Murawski, J. Naruniec, and K. Cho, “Augmentation for small object detection,” *arXiv preprint arXiv:1902.07296*, 2019.
- [14] X. X. Zhu, D. Tuia, L. Mou, G.-S. Xia, L. Zhang, F. Xu, and F. Fraundorfer, “Deep learning in remote sensing: A comprehensive review and list of resources,” *IEEE geoscience and remote sensing magazine*, vol. 5, no. 4, pp. 8–36, 2017.
- [15] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, “Yolov10: Real-time end-to-end object detection,” *Advances in neural information processing systems*, vol. 37, pp. 107 984–108 011, 2024.
- [16] C. Zhao, J. H. Hsiao, and A. B. Chan, “Gradient-based instance-specific visual explanations for object specification and object discrimination,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 9, pp. 5967–5985, 2024.
- [17] A. H. Ornek and M. Ceylan, “Haycamj: A new method to uncover the importance of main filter for small objects in explainable artificial intelligence,” *Neural Computing and Applications*, vol. 36, no. 18, pp. 10 791–10 798, 2024.
- [18] L. Liu, W. Ouyang, X. Wang, P. Fieguth, J. Chen, X. Liu, and M. Pietikäinen, “Deep learning for generic object detection: A survey,” *International journal of computer vision*, vol. 128, no. 2, pp. 261–318, 2020.
- [19] M. Carranza-García, J. Torres-Mateo, P. Lara-Benítez, and J. García-Gutiérrez, “On the performance of one-stage and two-stage object detectors in autonomous vehicles using camera data,” *Remote Sensing*, vol. 13, no. 1, p. 89, 2020.
- [20] R. Sapkota, M. Flores-Calero, R. Qureshi, C. Badgujar, U. Nepal, A. Poullose, P. Zeno, U. B. P. Vaddevolu, S. Khan, M. Shoman *et al.*, “Yolo advances to its genesis: A decadal and comprehensive review of the you only look once (yolo) series,” *Artificial Intelligence Review*, vol. 58, no. 9, p. 274, 2025.
- [21] J. Redmon and A. Farhadi, “Yolo9000: better, faster, stronger,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 7263–7271.
- [22] R. Huang, J. Pedoeem, and C. Chen, “Yolo-lite: a real-time object detection algorithm optimized for non-gpu computers,” in *2018 IEEE international conference on big data (big data)*. IEEE, 2018, pp. 2503–2510.
- [23] B. Karbouj, A. M. Altenbuchner, and J. Krueger, “Deep learning-based object detection for autonomous vehicles: A comparative study of one-stage and two-stage detectors on basic traffic objects,” *arXiv preprint arXiv:2602.00385*, 2026.
- [24] P. Mittal, “A comprehensive survey of deep learning-based lightweight object detection models for edge devices,” *Artificial Intelligence Review*, vol. 57, no. 9, 2024.
- [25] A. Wong, M. Famuori, M. J. Shafiee, F. Li, B. Chwyl, and J. Chung, “Yolo nano: A highly compact you only look once convolutional neural

- network for object detection,” in *2019 Fifth Workshop on Energy Efficient Machine Learning and Cognitive Computing-NeurIPS Edition (EMC2-NIPS)*. IEEE, 2019, pp. 22–25.
- [26] Z. Gong and D. Li, “Towards better object detection in scale variation with adaptive feature selection,” *arXiv preprint arXiv:2012.03265*, 2020.
- [27] K. Tong, Y. Wu, and F. Zhou, “Recent advances in small object detection based on deep learning: A review,” *Image and Vision Computing*, vol. 97, p. 103910, 2020.
- [28] Y. Bai, Y. Zhang, M. Ding, and B. Ghanem, “Sod-mtgan: Small object detection via multi-task generative adversarial network,” in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 206–221.
- [29] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, “Deformable detr: Deformable transformers for end-to-end object detection,” *arXiv preprint arXiv:2010.04159*, 2020.
- [30] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, “Path aggregation network for instance segmentation,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 8759–8768.
- [31] M. Tan, R. Pang, and Q. V. Le, “Efficientdet: Scalable and efficient object detection,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 10 781–10 790.
- [32] I. Suchý and M. Turčaník, “Review of large yolov8 and rt-detr energy efficiency on edge devices for real-time detection,” *Scientific Reports*, 2026.
- [33] M. F. Tariq and M. A. Javed, “Small object detection with yolo: A performance analysis across model versions and hardware,” *arXiv preprint arXiv:2504.09900*, 2025.
- [34] M. B. Muhammad and M. Yeasin, “Eigen-CAM: Class activation map using principal components,” in *2020 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2020, pp. 1–7.
- [35] F. Yang, H. Fan, P. Chu, E. Blasch, and H. Ling, “Clustered object detection in aerial images,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 8311–8320.
- [36] J. Zhang, J. Huang, X. Chen, and D. Zhang, “How to fully exploit the abilities of aerial image detectors,” in *Proceedings of the IEEE/CVF international conference on computer vision workshops*, 2019, pp. 0–0.
- [37] C. Li, T. Yang, S. Zhu, C. Chen, and S. Guan, “Density map guided object detection in aerial images,” in *proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops*, 2020, pp. 190–191.
- [38] C. Duan, Z. Wei, C. Zhang, S. Qu, and H. Wang, “Coarse-grained density map guided object detection in aerial images,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 2789–2798.
- [39] Z. Wei, C. Duan, X. Song, Y. Tian, and H. Wang, “Amrnet: Chips augmentation in aerial images object detection. arxiv 2020,” *arXiv preprint arXiv:2009.07168*.
- [40] S. Deng, S. Li, K. Xie, W. Song, X. Liao, A. Hao, and H. Qin, “A global-local self-adaptive network for drone-view object detection,” *IEEE Transactions on Image Processing*, vol. 30, pp. 1556–1569, 2020.
- [41] Z. Liu, G. Gao, L. Sun, and Z. Fang, “Hrdnet: High-resolution detection network for small objects,” *arXiv preprint arXiv:2006.07607*, 2020.
- [42] F. C. Akyon, S. O. Altinuc, and A. Temizel, “Slicing aided hyper inference and fine-tuning for small object detection,” in *2022 IEEE international conference on image processing (ICIP)*. IEEE, 2022, pp. 966–970.
- [43] C. Yang, Z. Huang, and N. Wang, “Querydet: Cascaded sparse query for accelerating high-resolution small object detection,” in *Proceedings of the IEEE/CVF Conference on computer vision and pattern recognition*, 2022, pp. 13 668–13 677.
- [44] B. Du, Y. Huang, J. Chen, and D. Huang, “Adaptive sparse convolutional networks with global context enhancement for faster object detection on drone images,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023, pp. 13 435–13 444.
- [45] X. Zhou, D. Wang, and P. Krähenbühl, “Objects as points,” *arXiv preprint arXiv:1904.07850*, 2019.
- [46] C. Liu, G. Gao, Z. Huang, Z. Hu, Q. Liu, and Y. Wang, “Yolc: You only look clusters for tiny object detection in aerial images,” *IEEE transactions on intelligent transportation systems*, vol. 25, no. 10, pp. 13 863–13 875, 2024.

Third C. Author Jr. (Member, IEEE), photograph and biography not available at the time of publication.